

PromCon EU 2026 · lightning talk

promql-cli, a full PromQL cli- only engine ✨ .. but WHY?!

JuanJo Ciarlante @jjo

title · 0:00–0:15

```
$ promql-cli query  
> .scrape $NODE 2 5s  
> rate(node_cpu<TAB>  
> .save incident.prom  
> .pinat node_load1  
> robust_zscore(...)  
> quit
```



Origin story

**12 hours. No Wi-Fi.
Lots of PromQL in mind... and **at hand**.**

PROD

P42

PASSENGER

a PromQL nerd

WI-FI

none

PROMETHEUS

left at the gate

CARRY-ON

incident.prom · 6h · 31,724 samples · 8.9 MB

ENGINE

upstream PromQL

STORAGE

in-memory

GATE

./promql-cli

What is it

A REPL for PromQL. The real engine, not a lookalike.

R

Read

`.scrape` an exporter,
`.load` a file,
`.prom_scrape_range` any
Prometheus API

E

Eval

the upstream `promql`
engine over in-memory
storage, experimental
functions on

P

Print

tables, `-o json`, or `.save`
it all back to a `.prom` file

L

Loop

tab completion, history,
`.pinat` to freeze time, `|`
`grep` to your shell

WHY #1

PromQL without a Prometheus

```
$ docker run -d -p 9100:9100 prom/node-exporter
$ promql-cli query -c \
  ".scrape http://localhost:9100/metrics 2 5s"
Scraped (1/2): +330 metrics, +1166 samples
Scraped (2/2): +0 metrics, +1166 samples (total: 2332)
> .pinat node_cpu_seconds_total # newest sample, ms-exact
> sort_desc(sum by (mode) (rate(node_cpu_seconds_total[1m])))
Vector (8 samples):
  [1] {mode="idle"}      => 1.6797
  [2] {mode="iowait"}    => 0.2380
  [3] {mode="user"}      => 0.0622
  [4] {mode="nice"}      => 0.0117
  ...
> .metrics | grep -c node_
287
```

No server, no TSDB, no YAML. Scrape twice and **rate()** just works.

Tab-completes metric names, labels and functions.

Debugging an exporter shouldn't need a Prometheus.

WHY #2

CI your exporters, with PromQL

```
# contract: one PromQL per line, non-empty = pass,
# the comment above it names the check
$ promql-cli check --scrape $NODE --count 2 \
  ci/node-exporter.contract.promql
PASS it is alive, and it tells us what it is
PASS counters actually count (needs >= 2 scrapes)
PASS ...and never go backwards between scrapes
PASS every CPU reports the same set of modes
FAIL cardinality budget: under 1000 node_* series
    count({__name__=~"node_.*"}) < 1000
    got: count({__name__=~"node_.*"}) = 1116, want < 1000
4 passed, 1 failed, 0 errors
$ echo $?
1                # CI goes red, and says why
```

promtool check metrics lints names.
promtool test rules runs on synthetic series.

Neither asserts PromQL on what your exporter **actually** serves: counters that count, no resets, a cardinality budget.

Exit 1: a check failed. Exit 2: the contract is broken. `--format junit` for your CI.

WHY #3

Prod in a file: the bug report reproducer

```
# on the ground, with Wi-Fi: one selector, 3 metrics, 6h
> .prom_scrape_range http://localhost:19090 \
  '{__name__=~"node_load1|\
  node_netstat_Tcp_RetransSegs|\
  kubelet_volume_stats_available_bytes"}' now-6h now 30s
+31724 samples (total: 3 metrics, 31724 samples)
> .pinat node_load1; .save incident.prom # 2 s, 8.9 MB

# ✈ 30,000 ft, Wi-Fi off
$ promql-cli query incident.prom
Pinned evaluation time: ...38:47.000Z (restored from incident.prom)
> sum(rate(node_netstat_Tcp_RetransSegs[5m]))
=> 13.45 # retransmits/s, whole fleet
> topk(3, rate(node_netstat_Tcp_RetransSegs[6h]))
[1] {...="172.16.16.75:9100"} => 3.66 # remember .75
[2] {...="172.16.16.76:9100"} => 2.04
[3] {...="172.16.16.79:9100"} => 1.72
```

Attach **incident.prom** to the issue: a reproducer, not a screenshot.

The pin travels **inside** the file (**# promql-cli: pinat=...**): maintainers get the exact samples at the exact time.

Plain PromQL, nothing new yet. That comes next.

WHY #4

An engine playground, no live Prometheus touched

```
# go.mod: build promql-cli against your Prometheus fork
replace github.com/prometheus/prometheus =>
    github.com/jjo/prometheus v0.0.0-20261005183247-88a37e36946b

# same incident.prom: TCP retransmits, 14 nodes, 6h
> topk(1, max_over_time(zscore(
    rate(node_netstat_Tcp_RetransSegs[5m]))[6h:30s]))
[1] {...="172.16.16.75:9100"} => 3.55 # under the ceiling...
> sqrt(count(node_load1) - 1)
=> 3.61 # ...which is sqrt(n-1)
> topk(1, max_over_time(robust_zscore(
    rate(node_netstat_Tcp_RetransSegs[5m]))[6h:30s]))
[1] {...="172.16.16.75:9100"} => 30.91 # 🦄
# and it has a name: topk() keeps the labels max() drops
```

New functions from a fork, on real prod data. Nothing deployed, nothing restarted.

.75 peaks at 17 retransmits/s, fleet median 2: median and MAD don't let the outlier hide itself.

11 new functions checked against plain-PromQL identities: 27 checks, worst relative error 6e-11.

Bonus catch: `promql.NewEngine` without `EngineOpts.Parser` silently disables experimental functions.

WHY #5

Alerts on prod data, before they page

```
> !cat rules/disk.rules.yaml
groups:
- name: disk
  rules:
  - record: pvc:days_to_full:6h
    expr: |
      sum by (persistentvolumeclaim, namespace) (
        time_to_threshold(kubelet_volume_stats_available_bytes[6h], 0)
      ) / 86400 > 0
  - alert: VolumeFullSoon
    expr: pvc:days_to_full:6h < 14
    labels: {severity: page}
> .rules rules/disk.rules.yaml
Rules: added 3 samples; 1 alerts
> sort(pvc:days_to_full:6h)           # the recording rule
[1] {...="mysql-data-mariadb-server-0"}   => 8.94
[2] {...="rabbitmq-data-rabbitmq-rabbitmq-0"} => 26782.59
[3] {...="storage-prometheus-stack-grafana-0"} => 6.20e+06
> ALERTS                               # the alert
[1] {alertname="VolumeFullSoon", alertstate="firing",
    ...="mysql-data-mariadb-server-0", severity="page"} => 1
```

An alert on a function your prod doesn't even run, fed by a recording rule, fires on prod data.

Before it ever pages anyone. **ALERTS** is a real series: query it, join it, **.save** it.

mariadb: ~9 days to full. Time to buy the DBA a coffee.

Evaluated at one instant (the pin): for: isn't modelled yet.

Rapid fire

`promql-cli mcp` · a PromQL sandbox for agents

`-o json` · pipe it anywhere

`.at / .pinat <metric>` · time travel

`--repl=prompt` · completion popup

`xjjo/promql-cli` · docker image

Because PromQL deserves a **REPL**.

github.com/jjo/promql-cli



github.com/jjo/promql-cli